

LEONARDO DA VINCI ESCUELA SUPERIOR DE ARTE MULTIMEDIAL

CARRERA:

Analista en sistema.

ASIGNATURA:

Diseño de Sistemas.

EXAMEN:

Parcial 2

AÑO LECTIVO:

2025

DOCENTES:

VALENZUELA, Horacio

EXAMEN PARCIAL 2 – Diseño de Sistemas

Formato de entrega:

- El parcial es de resolución individual.
- Se entrega en archivo comprimido que contenga el código y los diagramas correspondientes.
- El archivo comprimido llevará el nombre y apellido del alumno.

Ejemplo: *GamalielQuiroz.rar*

Desarrollo de la consigna:

Contexto:

La empresa ficticia "Excusas S.A." ha decidido modernizar aún más su sistema de gestión de excusas incorporando una **API REST** que permita a otras áreas del sistema (como Recursos Humanos o Control de Personal) registrar excusas, consultar el estado de las mismas, ver el historial de prontuarios, y administrar la línea de encargados en tiempo real.

El sistema ya está diseñado con una línea de encargados que manejan excusas usando el patrón **Chain of Responsibility**, y aplica principios como **Tell Don't Ask**, **DRY**, y **SOLID**. Cada excusa puede ser aceptada o rechazada, y puede generar efectos colaterales como envíos de emails o creación de prontuarios.

Endpoints

1. API RESTful con Spring Boot

Debés crear una API REST que permita:

- **GET /prontuarios**
Obtener el listado completo de prontuarios generados por los CEOs.
- **GET /empleados**
Obtener el listado completo de empleados.
- **POST /excusas**
 - Registra una excusa nueva asociada a un empleado existente.
 - La excusa será automáticamente evaluada por la línea de encargados y se generará un estado.
 - Se debe permitir enviar el motivo (como string), y el sistema debe convertirlo a un enum.
 - Si el motivo no existe, se debe devolver un **400 Bad Request**.
- **GET /excusas/{legajo}**
 - Devuelve la lista de excusas presentadas por un empleado específico.
- **GET /excusas**
 - Devuelve todas las excusas registradas en el sistema, con la posibilidad de aplicar filtros por fecha, motivo o encargado que la aceptó.
- **GET /excusas/busqueda?legajo={legajoEmpleado}&fechaDesde={una Fecha}&fechaHasta={otraFecha}**
 - Devuelve todas las excusas registradas en el sistema para un empleado de un legajo dado, con la posibilidad de aplicar filtros por fecha
- **GET /excusas/rechazadas**
 - Devuelve todas las excusas que fueron rechazadas.
- **POST /empleados**
 - Registra un nuevo empleado.
- **POST /encargados**
 - Permite agregar un nuevo encargado de manera dinámica, indicando qué motivos puede manejar (por ejemplo, un nuevo tipo como "Coach Motivacional").
- **PUT /encargados/modo**
 - Permite cambiar el **modo de evaluación** (**NORMAL**, **VAGO**, **PRODUCTIVO**) de un encargado existente.

- **GET /encargados**
 - Devuelve la configuración actual de la línea de encargados (orden y modo).
- **DELETE /excusas/eliminar?fechaLimite={unaFecha}**
 - Devolver cuántas excusas fueron eliminadas. Además, este proceso debe estar protegido para no ejecutarse por error: si no se proporciona el parámetro `fechaLimite`, la operación no debe realizarse y debe devolver un error claro.

Formato de datos: Usar JSON tanto para las peticiones como para las respuestas.

2. Persistencia

Utilizar JPA con una base de datos embebida (H2) para pruebas y MySQL en la rama productiva. Deberán persistirse.

- Empleados
- Excusas
- Prontuarios



Requisitos:

- Implementar la API solicitada utilizando spring boot + las dependencias que consideren necesarias para tal fin.
- Definir una arquitectura de tres capas: controladores, servicios y modelos.
- Usar DTOs para entrada y salida de datos.
- Usar **enums** para tipos de excusa y modos de trabajo.
- Persistencia en memoria (H2 o estructura simulada) o con JPA según el alcance del curso.
- Validaciones:
 - No se puede registrar una excusa sin un tipo válido.
 - No se puede eliminar excusas si no se pasa `fechaLimite`.
 - Otras que identifiquen
- Manejo de errores con HTTP status adecuados.
- Diagrama de clases actualizado
- Diagrama de clases de uso actualizado

- **Diagrama de arquitectura**

Consideraciones

- **Diseño limpio y desacoplado**, orientado a objetos, respetando TELL DONT ASK, PRINCIPIOS SOLID, POLIMORFISMO, ENCAPSULAMIENTO; DRY y otros.
- Uso de **interfaces y herencia** donde corresponda.
- Los servicios deben estar **bien separados del controlador** y del repositorio.
- Toda la lógica de negocio debe estar en **modelo, services, handlers o managers**, nunca en los controladores.
- Buen uso de las **anotaciones de Spring Boot** y manejo de excepciones.
- Los controladores deben retornar respuestas con códigos HTTP adecuados: Tanto las Solicitudes como las Respuestas deben estar homologadas
- Se valorará el uso de **DTOs**, mappers o adapters si es necesario.